Two Monolingual Transformer Language Models from Scratch: Hindi (Model H) and Nepali (Model L)

Swaraj Kumar

Language Models and Agents (Monsoon 2026) — Individual Project, Phases 1–3

Code: https://github.com/CL3-410/individual-project-swarajkumar001 Branches: `phase-1`, `phase-2`, `phase-3` (submission), `bonus-no-pos`

Checkpoints and corpora: Google Drive links in the repository `README.md`. September 2026

Abstract

I built two independent decoder-only Transformers from scratch, one for Hindi (higher-resource, "Model H") and one for Nepali (lower-resource, "Model L"). Nothing is shared between them except source code: separate corpora of 505,852,028 and 635,204,333 tokens, separate SentencePiece tokenizers (Unigram 8,000 and BPE 10,000), and separate weights. Both use the same architecture, so their non-embedding parameter count is identical at 18,915,328 and any difference in results comes from data and tokenizer rather than capacity. Each was pretrained on roughly 475M tokens on a Kaggle T4. Model H reaches test perplexity 27.41 against Model L's 46.68, but on bits-per-byte the order reverses (0.5382 against 0.4846), because Nepali's BPE packs 11.44 bytes into a token where Hindi's Unigram packs 8.88. In Phase 3 I finetuned each model on a synthetic comparative-reasoning corpus generated in its own language. Exact match rose from 0.00% to 35.15% (H) and 29.45% (L), but a forced choice between the entities the question names stays at or below the 41.67% chance rate, so what finetuning taught is the answer format, not the comparison. The same mistake — trusting one headline number — was available twice in this project, in opposite directions.

1 Introduction

Problem. Build two completely separate monolingual language models of about 25M parameters each, with no pretrained weights, no pretrained tokenizers and no HuggingFace Transformer classes: collect a corpus per language, train a tokenizer per language, implement a decoder-only Transformer, pretrain both, finetune both on reasoning data, and analyse what the attention heads learned.

Why this pair. Model H is Hindi, which has plenty of public text. Model L is Nepali, from the allowed lower-resource list. Both use Devanagari, which I chose deliberately: since the script is the same, script differences cannot explain any gap between the two models, and the comparison comes down to corpus size, corpus quality and tokenizer. Bhojpuri was my first Model L and was fully built and validated, but its corpus stopped at 84,217,032 tokens (16.8% of the 500M target). When the Nepali pipeline cleared its token targets I switched, and kept Bhojpuri's measured numbers in the Phase 1 report as history rather than deleting them.

Objective. Train both models on a comparable token budget and then find out what actually separates them — and, where a metric flatters one model, say why it does.

2 Phase 1: Corpora and Tokenizers

Objective. Two independent corpora of roughly 500M tokens each, at least 20% of training tokens from manual collection, cleaned, deduplicated, split, and tokenized by two separately trained tokenizers.

2.1 Sources

Hindi manual collection: a Hindi Wikipedia XML dump I extracted myself with WikiExtractor (the largest single manual source, 160,591 documents), Gadya Kosh (prose), Kavita Kosh (poetry), the `patrika.com` archive, and 67 `archive.org` items (Rajya Sabha / Lok Sabha transcripts, books). Downloaded: FineWeb-2 `hin_Deva` and AI4Bharat Sangraha (verified and unverified). I quarantined the HuggingFace-mirrored Hindi Wikipedia because it duplicates the dump I had already extracted. `scripts/manual_sites.json` records a robots.txt check per site; `bhaskar.com` and `amarujala.com` were ruled out because their robots.txt disallows automated agents.

Nepali manual: an `onlinekhabar` news scrape (91.6% of manual tokens), a self-parsed Wikipedia dump, BBC Nepali, Wikibooks, Wiktionary, Ekantipur, Ratopati, Nagarik, and OCR of archive.org books. Downloaded: AI4Bharat Sangraha and IndicCorpV2. Nepali gates on two *absolute* targets instead of a percentage — manual $\geq$ 100M tokens, downloaded $\geq$ 400M — and both cleared (167,787,251 and 467,417,082).

2.2 Cleaning and normalisation

Normalisation is four operations, in `scripts/clean_lib.py:75`: NFC normalise, strip zero-width and control characters, collapse whitespace runs, collapse blank-line runs. The stripping regex removes U+200E, U+200F, U+FEFF and

C0/C1 controls but **deliberately keeps ZWNJ and ZWJ**, because in Devanagari those change what is rendered — they are text, not formatting noise. NFC matters for deduplication too: the same visible Devanagari string can be stored precomposed or as a base plus a combining mark, and without normalisation those two forms compare as different strings and both survive.

Filters, all threshold-based: minimum 200 characters, Devanagari character ratio ≥ 0.5, malformed-Unicode ratio (U+FFFD) ≤ 0.01, and a langdetect check at `min_prob` 0.85. The language check is three-valued on purpose — true, false, or *None* when the text is too short or ambiguous — so "undetermined" is not silently treated as "wrong language". Hindi also runs a Bhojpuri reject filter, which is needed because Bhojpuri shares the script and much of the vocabulary. Nepali scores every document, manual sources included, with fastText `lid.176` at stage S04.

Table 1: Hindi cleaning. 95.9% of raw characters survive; the dominant drop is short documents, not language or encoding problems.

Stage	Documents	Characters	Drop reason	Documents
Manual (raw)	229,011	540,367,035	too short (<200 chars)	63,132
Downloaded (raw)	544,595	1,299,990,112	exact/near duplicate	2,983
Raw total	773,606	1,840,357,147	langdetect mismatch	1,778
Cleaned	704,103	1,764,598,107	likely Bhojpuri	1,365
Retention	—	95.9% of chars	low Devanagari ratio	228
			malformed Unicode	17

2.3 Deduplication — the two pipelines differ

I want to state this plainly rather than smooth it over. **Hindi does not use MinHash.** Its `dedup_key()` (`scripts/clean_lib.py:153`) collapses whitespace, strips punctuation, lowercases and takes a SHA-1 hash, so it catches exact duplicates and whitespace or formatting variants of the same document, but not paraphrase-level similarity. **Nepali does**: stage S08 matches document ids exactly first, then runs MinHash with LSH — 100 permutations, 20 bands of 5 rows, 5-character shingles, Jaccard threshold 0.55, seed 20260817 — and removed 2,350 exact and 49,734 near duplicates. Where a duplicate pair straddled the manual and downloaded lanes (21,273 pairs) the manual copy was kept, which protects the manual token share. Nepali also has a decontamination stage (S09) that MinHashes train against the held-out sets and removes matches *from train only*; it removed 17,111 documents and a rescan found 0 remaining matches. Hindi has no equivalent stage. This asymmetry is a real limitation of the Hindi pipeline and the first thing I would fix.

2.4 Splits

Both are document-level, never chunk-level: if chunks of one article land on both sides of a split, the test score partly measures memorisation of text the model has already seen. Hindi is 96/2/2 with seed 42, shuffle then slice. Nepali is 85/7.5/7.5 with seed 20260817 by a deterministic hash of the document id, and it holds the evaluation sets out at S07 — *before* the quality filters run — so the heuristic and perplexity filters curate only the training pool and cannot quietly make the test set easier. `split_stats.json` records `zero_doc_id_overlap: true` and per-split manual ratios of 0.2742 / 0.2371 / 0.2311. The ratios differ between languages because Nepali has far more documents, so 7.5% still leaves over 519M training tokens.

2.5 Tokenizers and vocabulary choice

Two tokenizers trained separately, no vocabulary shared. I picked the vocabulary size by sweeping and measuring on held-out text rather than by rule of thumb. Fertility here is tokens per whitespace word: lower means the tokenizer needs fewer tokens to write the same text, so the model sees more text per step.

Table 2: Measured tokenizer sweep. Nepali's byte-fallback column is zero at *every* size — that is the defect, not a good score, and it is exactly why its unknown rate is non-zero.

Language	Vocabulary	chars/token	fertility (tok/word)	unk rate	byte fallback
Hindi	5,000	3.3230	1.4961	0.000000	0.018301
Hindi	**8,000**	**3.5622**	**1.3956**	**0.000000**	**0.019613**
Hindi	10,000	3.6601	1.3583	0.000000	0.020213
Nepali	5,000	3.7882	1.6988	0.002956	0.000000
Nepali	6,000	3.9410	1.6329	0.003075	0.000000
Nepali	8,000	4.1725	1.5424	0.003256	0.000000
Nepali	**10,000**	**4.3410**	**1.4825**	**0.003387**	**0.000000**

Hindi uses SentencePiece **Unigram at 8,000**. Fertility still improves at 10,000, but the gain from 8,000 to 10,000

is smaller than from 5,000 to 8,000, and 2,000 extra vocabulary rows cost about 1M embedding parameters out of a 25M budget. Nepali uses **BPE at 10,000**, the best fertility in its sweep at 1.4825.

The mistake is in the last column. Hindi's Unigram model has byte fallback, so a character it has never seen is still encoded as bytes and its unknown rate is exactly 0. Nepali's BPE was trained without byte fallback, so anything outside the vocabulary becomes `<unk>` and is unrecoverable — 0.003387 on held-out text. I only learned what this really costs in Phase 3 (§4.1).

2.6 Phase 1 results and observations

Hindi: 505,852,028 tokens over 704,101 documents, manual share 30.66% (155,104,651 tokens), clearing the 20% floor. Nepali: 635,204,333 tokens over 1,918,265 documents, manual share 26.41%. Every count is exact — each document in each split was actually tokenized and counted, not sampled and extrapolated. Nepali's funnel runs 2,655,504 documents in to 2,126,334 after dedup, with language ID removing only 0.22%, which is what I would expect from sources that were already Nepali-labelled.

One bookkeeping detail, because two different numbers appear for the same split: Phase 1 reports 485,280,741 tokens for Hindi train, Phase 2 reports 485,956,678. The difference is 675,937, exactly the number of training documents — packing appends one end-of-document token per document. The same holds for Nepali (1,520,000). Nothing is inconsistent; the counts are taken at different stages.

3 Phase 2: Architecture, Pretraining and Evaluation

Objective. Implement a decoder-only Transformer from primitive PyTorch layers, pretrain both models independently with resume-capable checkpointing, and evaluate them as language models.

3.1 What the forward pass does

Token ids go through an embedding table of shape (vocabulary $\times$ 512), a learned absolute positional embedding of shape (512×512) is added, and embedding dropout is applied. Each of the 6 blocks is pre-norm: LayerNorm, then multi-head causal self-attention, added back to the residual stream; LayerNorm, then a feed-forward network of $512 \to 2048 \to 512$ with GELU, added back again. A final LayerNorm precedes the output projection to vocabulary logits, which is tied to the input embedding. The loss is cross-entropy between the logits at position t and the token at $t+1$.

Inside attention (`hindi/model/transformer.py:69`) I keep W_q, W_k, W_v as three separate `nn.Linear` layers rather than one fused matrix, because it makes the three roles visible in the code. Each projection maps $(B, T, 512)$ to $(B, T, 512)$; a `view` to $(B, T, 8, 64)$ and a `transpose` to $(B, 8, T, 64)$ turn that into 8 independent 64-dimensional heads. Scores are $QK^\top/\sqrt{d_k}$ with $d_k = 64$, so the divisor is 8. The scaling is needed because the dot product of two d_k-dimensional vectors grows with d_k; without it the softmax input variance grows, the softmax saturates, and gradients through it get very small. The causal mask is a `torch.triu(..., diagonal=1)` boolean buffer registered with `persistent=False` (it is fully determined by `block_size`, so it does not belong in a checkpoint) and is applied with `masked_fill(-inf)` *before* the softmax. After the softmax, attention dropout; then $(B, 8, T, T) \times (B, 8, T, 64)$ gives the per-head output, which is transposed back, made contiguous (transpose only permutes strides, while `view` needs a contiguous buffer), reshaped to $(B, T, 512)$ and passed through W_o with residual dropout.

Causal masking, verified rather than assumed. `hindi/tests/test_model.py:51` perturbs every token after position t, for every t, and checks the logits at positions $\leq t$. Maximum observed change: **0.000e+00**. It is exactly zero, not merely small, because softmax$(-\infty)$ is exactly 0 and $0 \times v$ is exactly 0 in floating point. The same test asserts the converse — the logits *at* the perturbed positions must move — otherwise a model that ignored its input entirely would pass. Both test suites pass 9/9 independently.

3.2 Design choices I had to justify

Depth over width. At $d = 512$ each block costs about 3.15M parameters, so 6 blocks plus the embedding table lands at 23.3M; 7 blocks would overshoot to 26.4M. I spent the budget on 6 blocks rather than a wider 4-block model because attention needs several rounds of composition to build multi-hop dependencies. **Learned absolute positions** over sinusoidal, because the model is trained and evaluated at one fixed context length where sinusoidal extrapolation buys nothing. The consequence I accepted: `block_size` 512 is a hard ceiling — there is no embedding row for position 512, so longer sequences need retraining that table. **Weight tying** saves $8000 \times 512 = 4{,}096{,}000$ parameters for Hindi, about 15% of the model. **Pre-norm**, because normalising the input to each sublayer keeps the residual path clean and the gradient better behaved than post-norm at this depth.

3.3 Training setup

AdamW ($\beta = 0.9, 0.95$), weight decay 0.1 applied only to parameters with `dim` ≥ 2 — decaying LayerNorm gains and biases shrinks them towards zero for no benefit. Learning rate 6e-4 peak with 500 warmup steps then cosine decay

Table 3: Both models. Architecture is identical, so the non-embedding parameter count is identical and differences trace to data and tokenizer.

	Model H (Hindi)	Model L (Nepali)
Vocabulary / tokenizer	8,000 Unigram	10,000 BPE
Layers / heads / d / d_{ff} / context	6 / 8 / 512 / 2048 / 512, dropout 0.1	
Embedding parameters	4,358,144	5,382,144
Non-embedding parameters	**18,915,328**	**18,915,328**
Total parameters	23,273,472	24,297,472
Steps completed (of 15,000)	14,520	14,440
Training tokens	475,791,360	473,169,920
Evaluated checkpoint	step 14,500	step 14,000
Test cross-entropy (nats)	3.3109	3.8433
Test perplexity	**27.41**	46.68
Test bits-per-byte	0.5382	**0.4846**
Bytes per token	8.875	11.441

to 6e-5. Gradient clipping at 1.0, applied after `scaler.unscale_()` so the clip sees true gradient magnitudes rather than scaled ones. Effective batch is $32 \times 2 \times 512 = 32{,}768$ tokens per step (micro-batch 32, gradient accumulation 2); 15,000 steps would be about 491.5M tokens, roughly one epoch. Seeds 20260902 (H) and 20260903 (L).

Precision: the config requests bfloat16 and the trainer resolves it against the device, falling back to float16 with a `GradScaler` where bfloat16 is unsupported. The recorded Kaggle run logs `device=cuda dtype=torch.bfloat16` on the Tesla T4, so **pretraining actually ran in bfloat16**; the float16 path is what Phase 3 used (§4.2). I mention it because my own earlier notes had this the wrong way round, and the kernel log is the authority.

Checkpoints carry model weights, optimizer state, scheduler state, current step, best validation loss, tokens consumed, both RNG states and the full run config, so an interrupted session resumes exactly rather than approximately. I verified resume across three consecutive sessions before starting the real runs.

Both runs stopped short of the plan. An 11-hour wall-clock guard fired at step 14,520 (H) and 14,440 (L) out of 15,000, at roughly 12.0k tokens/s. Because checkpoints are saved on best validation loss, the evaluated checkpoints are steps 14,500 and 14,000 — so the models are 3% short of their intended token budget, not broken by it. Final evaluation ran on CPU for both models deliberately, so that temperature sampling drew from the same RNG in both cases.

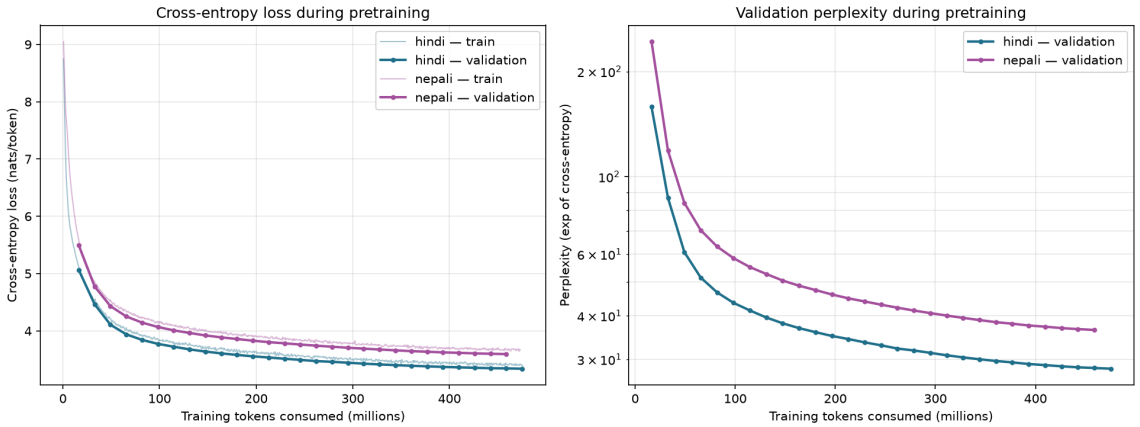


Figure 1: Training and validation loss against tokens consumed. Both curves are still descending when the wall-clock guard fires, which is why I describe the runs as budget-limited rather than converged.

3.4 Perplexity and bits-per-byte disagree

This is the Phase 2 result I care most about. Scored over 1,999,872 tokens per model, Model H's test perplexity is 27.41 and Model L's is 46.68 — a 41% gap that reads as "the lower-resource model is much worse". But perplexity is per *token*, and a token is not the same unit in the two models. Converting to bits and dividing by bytes:

$$\mathrm{BPB}_H = \frac{4.7766}{8.875} = 0.5382, \qquad \mathrm{BPB}_L = \frac{5.5447}{11.441} = 0.4846.$$

On bits per byte of UTF-8 text, Model L is marginally *ahead*. Both numbers are correct; they measure different things. Nepali's BPE packs more text into each token, so its tokens are individually harder to predict but each one buys more characters. Most of the apparent perplexity gap is tokenizer granularity, not model quality. The caveat I should state: BPB is deflated for both models because Devanagari costs three UTF-8 bytes per character, so these values are not comparable to BPB figures for Latin-script corpora.

3.5 Generation and attention

Table 4: Generation against held-out references, four decoding settings. Repetition is the fraction of repeated 4-grams.

	chrF++		BLEU-4		repetition		Distinct-1	
Decoding	H	L	H	L	H	L	H	L
greedy	12.49	14.48	1.96	2.37	0.598	0.577	0.172	0.179
temperature 0.5	14.33	16.54	1.80	1.88	0.212	0.190	0.275	0.345
temperature 1.0	**16.29**	**18.05**	1.01	1.69	0.002	0.001	0.520	0.634
temperature 1.5	14.67	16.13	0.16	0.78	0.000	0.000	0.702	0.793

Greedy decoding degenerates in both models: about 58–60% of generated 4-grams are repeats and the samples visibly loop. That follows from what greedy decoding is — the argmax is deterministic, so once the model enters a state it has seen before it must produce the same continuation and cannot leave the loop. Temperature 1.0 removes the repetition and simultaneously peaks chrF++, so on these models diversity costs no fidelity up to that point. BLEU is near the floor everywhere and I would not trust it here: open-ended continuations have no reason to share 4-grams with one reference continuation. chrF++ is the more informative of the three because it works on character n-grams, which suits a morphologically rich Devanagari language where a single suffix change breaks a whole word match.

Attention was measured with my own toolkit. Mean entropy is 3.2047 (H) and 3.0898 (L) against a uniform-attention baseline of 5.2565, so both models are about 2 nats sharper than attending everywhere equally — the baseline is what makes the raw number mean anything. Mean attention distance is 62.36 and 54.51 tokens. The layer structure is the interesting part: in both models the sharpest head per layer gets steadily sharper and more local from layer 0 to layer 3, where it becomes a previous-token head (entropy 0.06 at distance 1.3 for H, 0.14 at 1.1 for L), then spreads out again at layer 5 (distance 151.4 and 81.3). Two models with different languages, tokenizers and seeds independently arrived at the same organisation, which I did not expect at 23M parameters.

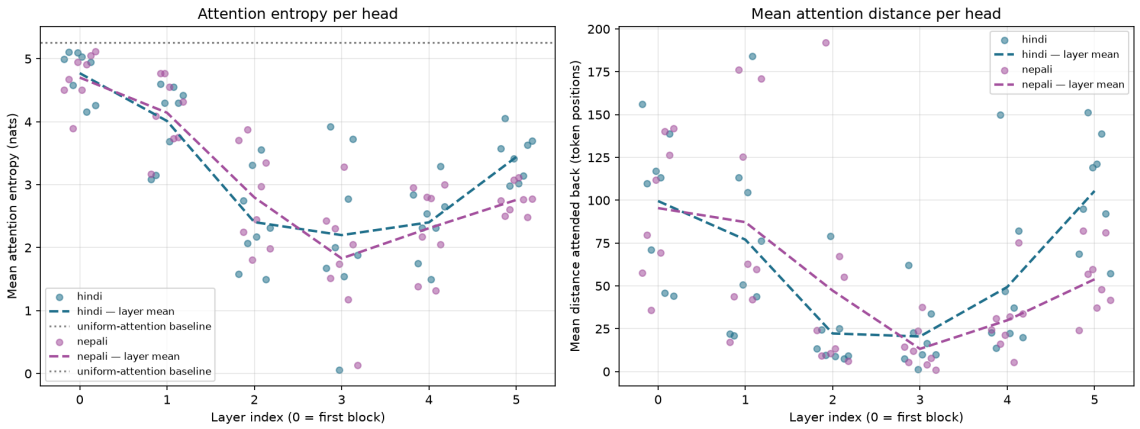


Figure 2: Per-head attention entropy and mean attention distance by layer. The layer-3 dip is the previous-token head in both models.

4 Phase 3: Reasoning Finetuning and Attention Comparison

Objective. Finetune each pretrained model on synthetic comparative-reasoning data written in its own language, measure what changed, and compare attention before and after.

4.1 The synthetic dataset

Both corpora are generated programmatically by `{hindi,nepali}/reasoning/generate.py`, so every ground-truth label is known by construction and nothing is downloaded. Four task families, 5,000 training examples each: `compare2`

(one comparison, then a question about the pair), `transitive3` (two chained comparisons — the answer needs the chain), `numeric2` and `numeric3` (compare two or three measured quantities). Splits are 20,000 / 2,000 / 2,000 per language. A Hindi test example the finetuned model gets right: ध्यान दीजिए: प्रवीण वरुण से हल्का है। भारी कौन है? — gold वरुण, model वरुण.

Leakage control uses both defences the brief suggests, together: 40/10/10 disjoint entity names and 14/4/4 disjoint sentence templates, so a test question is about unseen people phrased in an unseen way. This is *asserted* in `main()` rather than assumed, and recorded in `dataset_stats.json` as `entity_names_disjoint_across_splits: true` and `templates_disjoint_across_splits: true`. The two languages share no sentence, entity name or template.

Two defects I found while building this, both of which would have invalidated the results. First, a gender-agreement leak: Hindi adjectives inflect, so सबसे छोटी कौन है? asked about one woman and two men has only one grammatically possible answer — 181 of 400 examples were solvable without comparing anything. Fixed by drawing all entities in an example from one gender. Second, Model L's vocabulary contains *no Devanagari digit at all*, and its BPE has no byte fallback (§2.5), so every numeral encoded to `<unk>` at a 4.8% corpus rate. Nepali numerals are now written with Arabic digits, which the vocabulary covers and which modern Nepali prose uses anyway. Both corpora now tokenize at a 0.000000 unknown rate. The second defect is the clearest example in this project of a Phase 1 decision only becoming visible two phases later.

4.2 Protocol

Each model starts from its own Phase 2 checkpoint with its own Phase 1 tokenizer held fixed — the finetuner *asserts* that the tokenizer's vocabulary size matches the checkpoint rather than trusting the config. Loss is computed on answer tokens only, using `ignore_index=-1` on the prompt positions, so the model is not rewarded for memorising question text. Batches are built by length bucketing rather than padding, which is not cosmetic: with learned absolute positions, padding at the front would shift every real token's position index and change what the model sees.

Hyperparameters: 2 epochs, batch 32, peak learning rate 6e-5 (one tenth of pretraining) with 100 warmup steps, min 6e-6, weight decay 0.1, grad clip 1.0, seed 20260914, float16 with a `GradScaler` because the Kaggle T4 is compute capability 7.5. The reduced learning rate is the important one: a pretraining-sized step overwrites the language model before the task is learned.

Run A versus run B. My first run evaluated every 100 steps and picked its best checkpoint at step 100, which is exactly where warmup ends — the grid was too coarse to locate the real optimum. Moving to a 20-step grid, with nothing else changed, raised Model L from 14.90% to 29.45% exact match and Model H from 32.70% to 35.15%. Run A is kept in `report/phase3/run_a_40names/`.

4.3 Two metrics, and why both are needed

Exact match after greedy decoding is the honest end-to-end number, but it floors at zero on a model that has never seen the prompt format, so it cannot separate "learned nothing" from "learned the task but not the format". So I also score a **forced choice** among the entities the question actually names, using the length-normalised mean log-probability per answer token (without normalising, shorter names win for being shorter). Chance is computed per example as the mean of $1/n_{\text{candidates}}$, which comes to 41.67% across the test set — two-candidate families give 50%, `numeric3` and `transitive3` give 33.3%.

Table 5: Reasoning results on 2,000 held-out test questions per model. Chance for forced choice is 41.67%.

	H pretrained	H finetuned	L pretrained	L finetuned
Exact match	0.0000	**0.3515**	0.0000	**0.2945**
Forced choice	0.3980	0.3635	0.3880	0.3930
Answer-token loss	5.2439	0.9408	5.5367	1.4351
Answer-token perplexity	189.40	2.56	253.84	4.20

4.4 Results and error analysis

Exact match going from 0 to 35.15% and 29.45% looks like success, and answer token perplexity falling from 189 to 2.6 shows the models did learn something substantial. But forced choice sits at or below chance for both, and per task the agreement with chance is close to exact: Model H scores 50.40% and 47.00% on the two two-candidate families (chance 50%) and 33.20% on three-candidate `numeric3` (chance 33.3%). The honest reading is that finetuning taught the output format — answer with a name from the prompt, in this shape — and not the comparison.

The one place the models leave chance, they leave it *downward*: `transitive3` scores 12.40% (H) and 13.40% (L) against 33.3%. The reason is measurable. On *"A > B. B > C. Who is largest?"* Model H answers with the middle entity 72.6% of the time and the first entity 0.0% of the time, while the gold answer is the first entity 47.4% and the

last 52.6% — the middle entity is never correct. B is the only name mentioned twice, so the model is selecting the most repeated name instead of following the chain. Model L does the same thing more weakly (41.8% middle) but has a second problem: it answers with a name that is not in the prompt at all 28.8% of the time on this task, against 2.0% for Model H. Across all four tasks Model H's off-prompt rate is 0.0–7.2% and Model L's is 19.2–28.8%, which is a concrete resource-tier difference — the higher-resource model is better grounded in the text it was given.

Both models overfit quickly: validation loss bottoms at **step 60** in both and rises monotonically after, to 3.5188 by step 1,250, while training loss falls to 0.4441. With 40 training names and 14 templates, memorising the answer prior is easier than learning to copy from the prompt. Saving on best validation loss is what kept the evaluated checkpoints usable — the evaluated checkpoint is step 60, not the badly overfit final one, so the overfitting did not contaminate the reported numbers.

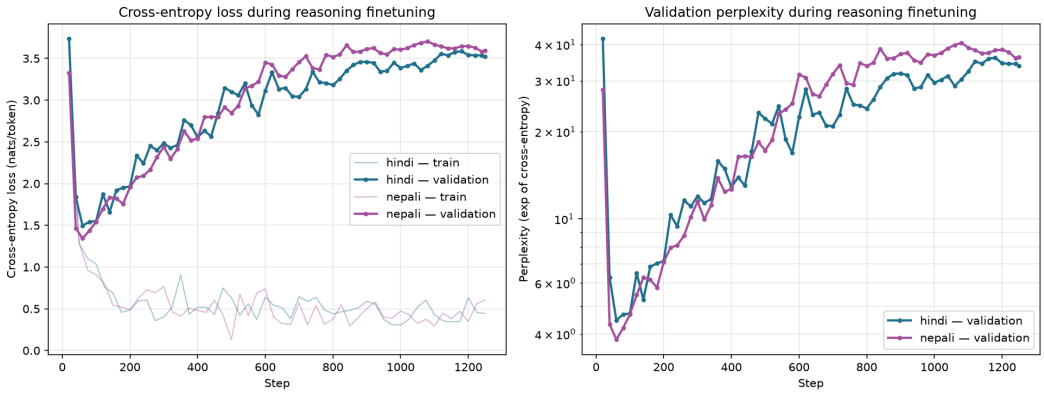


Figure 3: Finetuning loss. Validation (dashed) bottoms at step 60 in both models while training loss keeps falling — the gap is the answer prior being memorised.

4.5 Attention, pretrained versus finetuned

Both checkpoints were run on the *same* 16 held-out reasoning prompts, all of one token length per language, so that prompt text and sequence length cannot masquerade as a finetuning effect. Model H: mean entropy 1.9102 → 1.8691, mean distance 10.303 → 10.360, uniform baseline 2.9545 at $T = 46$. Model L: entropy 1.8327 → 1.7833, distance 9.415 → 10.157, baseline 2.8061 at $T = 39$. So finetuning made attention sharper and longer-range at the same time in both models, most clearly in Model L, whose mean distance rises 0.74 positions on a 39-token prompt. The heads that moved most are in the middle layers — H's largest shifts are L1H5 (−0.27 entropy) and L3H1 (−0.23 entropy, +2.59 distance); L's largest distance shift is L3H3 at +3.71 — which is where Phase 2 found the previous-token heads. I should be careful about what this shows: it identifies heads that changed, not heads that carry the comparison. Ablating L3H1 and re-scoring would be the direct test and I did not run it.

5 Cross-Phase Findings

How the work built up. Phase 1 decisions did not stay in Phase 1. The tokenizer choice made in week one set the bytes-per-token ratio that reversed the Phase 2 ranking, and the missing byte fallback — invisible in Phase 1, where the unknown rate looked like a tiny 0.003 — became a 4.8% unknown rate on Phase 3 data because that data contained numerals. Phase 2's attention toolkit was reused unchanged on Phase 3 checkpoints, which is only possible because the finetuned checkpoints keep the pretraining payload format.

What the results indicate. The two models are separated much less than perplexity suggests. On bits per byte they are within 0.06 of each other, with Model L slightly ahead; their attention organisation is nearly the same; their reasoning scores are both at chance. Where they genuinely differ is grounding: Model L invents names that were not in the prompt on 19.2–28.8% of examples against Model H's 0.0–7.2%, and on `numeric2` Model H never does it at all. That is the kind of difference I would expect from data quality and tokenizer granularity rather than from a capacity gap, which is consistent with the two models having identical non-embedding parameter counts.

Bonus ablation (branch `bonus-no-pos`, not claimed on the submission branch). Retraining Model H with `pos_encoding: none`, one variable changed, cost only 5.8% test perplexity (27.41 → 29.01) and 0.0092 BPB, far less than permutation invariance alone would predict, while attention entropy rose from 3.2047 to 3.6005 and the local-head fraction rose from 0.0625 to 0.2083. The usual explanation is that causal masking itself leaks position — position t can attend to t keys, so the number of visible tokens encodes position. I want to state that as an explanation consistent with the evidence, not as something this experiment proves; proving it needs a probe I did not run. Also:

the ablated run got a faster GPU and finished all 15,000 steps, so it is the better-trained of the two and no wall-clock comparison between them is meaningful.

6 Challenges and Lessons Learned

Bhojpuri did not reach the token target. I built the whole Bhojpuri pipeline and it passed every validation check, but public Bhojpuri text ran out at 84.2M tokens. Rather than pad the corpus with low-quality scraping I moved Model L to Nepali and kept Bhojpuri's numbers as a documented shortfall. The lesson was to check source availability before building the pipeline, not after.

The Nepali dedup stage took over 30 hours. S08 MinHash over about 2.2M documents was impractical on my laptop inside the deadline, so it ran on a separate machine and only output artifacts came back. That means the S08–S14 entries in `log.md` are reconstructed from artifacts rather than live logs, and they are flagged inline as such.

Two training runs hit the 11-hour guard. I set a wall-clock exit so a Kaggle session could not die without a usable checkpoint, and it fired at step 14,520 and 14,440 out of 15,000. It cost about 3% of the token budget and was the right trade.

A checkpoint-selection grid that was too coarse. Evaluating every 100 steps put the best checkpoint at the exact step where warmup ended. That is not a subtle bug — it is an experiment-design mistake, and it changed Model L's headline number by 14.55 points once fixed.

Technically, the three things I did not know before. How an additive $-\infty$ mask makes future leakage exactly zero rather than approximately zero; why per-token perplexity cannot be compared across tokenizers, and what to do instead; and that a metric can move a long way ($0 \rightarrow 35\%$ exact match) while the underlying ability does not move at all.

7 Reflection

The thing I got wrong at the start was treating tokenizer training as a preprocessing step to get past. I chose Nepali BPE on fertility alone, and the sweep table I produced actually showed the problem — byte fallback 0.000000 at every vocabulary size — but I read that column as a good score instead of as a missing capability. It cost me a 4.8% unknown rate on the Phase 3 data and a regeneration of the entire Nepali reasoning corpus. What I would do differently is not complicated: enable byte fallback and re-sweep.

The second thing I misunderstood was what perplexity means across two models. When I first saw 27.41 against 46.68 I wrote in my notes that the lower-resource model was much worse, and I was ready to attribute it to corpus quality. Computing bits per byte changed the conclusion, and the arithmetic is simple enough that the error was purely mine for not doing it earlier. The same shape of mistake was waiting in Phase 3: $0 \rightarrow 35.15\%$ exact match is a real measurement and a misleading summary, and the only reason I know that is that I built a chance-calibrated second metric instead of stopping at the flattering one.

What I am most confident about is the code I wrote myself — the attention implementation, the causal-mask test and the evaluation toolkit — because each of those was written to be checked rather than trusted, and the causal-mask test in particular is the one place in the project where I can point at a number (0.000e+00) and explain exactly why it cannot be anything else.

Limitations. The models do not perform the reasoning task: forced choice is at or below chance, and nothing here should be read as evidence that they reason. The finetuning distribution is too narrow in two independent ways — 40 entity names and 14 templates — and widening either is the change I would make first. Every result comes from a single seed per model, so the Model H / Model L differences are not bracketed by run-to-run variance. The attention analysis is correlational. Hindi has no near-duplicate or decontamination stage, so its test score is less protected than Nepali's. And both pretraining runs are 3% short of their planned budget with loss still descending, so neither model is converged.

8 Conclusion

Two independent 23–24M parameter decoder-only Transformers were built from scratch and trained to about 475M tokens each, one on 505.9M tokens of Hindi and one on 635.2M tokens of Nepali, with separate corpora, tokenizers and weights throughout. Model H ends at 27.41 test perplexity and Model L at 46.68, but at 0.5382 and 0.4846 bits per byte, and the reversal is a tokenizer effect rather than a quality effect. Finetuning both on self-generated comparative-reasoning data raised exact match from 0 to 35.15% and 29.45% while forced-choice accuracy stayed at chance, so the models learned the answer format and not the comparison — and the transitive failure has a specific, measured cause, which is that the model picks the name mentioned twice. The most useful outcome for me was learning where single-number metrics stop being trustworthy, which happened twice in this project in opposite directions.

References

1. Vaswani et al., *Attention Is All You Need*, NeurIPS 2017 — the decoder-only architecture, scaled dot-product attention and the $\sqrt{d_k}$ scaling implemented in `model/transformer.py`.
2. Kudo & Richardson, *SentencePiece: A simple and language independent subword tokenizer*, EMNLP 2018 — the tokenizer library used for both models.
3. Kudo, *Subword Regularization*, ACL 2018 — the Unigram algorithm used for Hindi.
4. Sennrich et al., *Neural Machine Translation of Rare Words with Subword Units*, ACL 2016 — BPE, used for Nepali.
5. Loshchilov & Hutter, *Decoupled Weight Decay Regularization*, ICLR 2019 — AdamW; and *SGDR*, ICLR 2017 — cosine schedule.
6. Hendrycks & Gimpel, *Gaussian Error Linear Units*, 2016 — the feed-forward non-linearity.
7. Broder, *On the resemblance and containment of documents*, 1997 — MinHash, used in the Nepali S08 dedup and S09 decontamination stages.
8. Papineni et al. (BLEU, ACL 2002); Popović (chrF, WMT 2015); Lin (ROUGE, 2004) — generation metrics, computed with `sacrebleu` and `rouge-score`.
9. Joulin et al., *fastText* `lid.176` language identification model — Nepali stage S04; `langdetect` for the Hindi pipeline.
10. Data sources: HuggingFaceFW FineWeb-2 (`hin_Deva`); AI4Bharat Sangraha and IndicCorpV2; Wikimedia dumps extracted with WikiExtractor; Gadya Kosh, Kavita Kosh, `patrika.com`, `onlinekhabar`, BBC Nepali and `archive.org` items, collected as described in §2.1.